



Setting Up Full Multigrid

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Most of the time in a geodynamics model goes into solving the Stokes equations, and most of that goes into the velocity block. How that block is preconditioned decides how long a model takes to run, and it is one of the few settings where the right choice is worth an order of magnitude.

1. WHAT MULTIGRID DOES

An iterative solver reduces error unevenly. Simple relaxation methods — the smoothers — are good at removing error that varies rapidly from cell to cell, and poor at removing error that varies smoothly across the whole domain. The smooth part is the expensive part: on a fine mesh it takes many sweeps to move information from one side of the model to the other.

Multigrid removes that part somewhere else. Error that is smooth on a fine mesh is *not* smooth on a mesh twice as coarse, where it spans half as many cells, so a few sweeps on the coarse mesh remove what the fine mesh could not. The method is a recursion: smooth on the fine mesh, transfer what is left to a coarser one, solve there (by recursing again), transfer the correction back, smooth again. Each level handles the band of error it can see cheaply, and the work per unknown stops growing as the model gets bigger.

Two things have to be supplied: the coarse levels, and the operators that move between them.

2. TWO WAYS TO BUILD THE COARSE LEVELS

Algebraic multigrid builds them from the matrix. It inspects the operator's connectivity, groups unknowns that are strongly coupled, and constructs coarse levels and transfer operators from that grouping alone. It needs nothing from the mesh, which is why it works everywhere and is the sensible default. PETSc's implementation is GAMG, and it is what Underworld uses when it has nothing better.

Geometric multigrid uses actual coarser meshes. If the fine mesh was built by refining a coarse one, those coarser meshes already exist, and the transfers follow from the refinement relation: each fine node either coincides with a coarse node or lies inside a known coarse cell. In PETSc this is `pc_type=mg`, and run as a *full* multigrid cycle — starting on the coarsest level and working up, rather than starting fine — it is what we call FMG.

The difference matters when the operator's connectivity is a poor guide to the geometry, which is exactly what a strong viscosity contrast produces. The algebraic method sees a matrix whose couplings are dominated by the stiff region and groups unknowns accordingly; the geometric method does not care, because it was told the grids.

3. MAKING A MESH THAT HAS A HIERARCHY

A mesh only carries a hierarchy if it was built with one. That is the `refinement` argument:

```
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)
len(mesh.dm_hierarchy)      # 3: the base and two refinements
```

The mesh you get back is the finest level. The coarser ones are kept alongside it, and they are what the preconditioner uses. Build the same mesh at `cellSize=0.0125` with no refinement and you get a mesh of about the same resolution with no hierarchy at all — and no geometric multigrid.

The solver picks it up on its own:

```
stokes = uw.systems.Stokes(mesh)
stokes.preconditioner      # "auto" by default
stokes.solve()
```

"auto" uses geometric multigrid when `len(mesh.dm_hierarchy) > 1` and GAMG when it does not. You can also ask directly, with `"fmg"` or `"gamg"`. Two behaviours are deliberate: "auto" only ever adds geometric multigrid to an untouched configuration, so it will not overwrite preconditioner options you have set yourself; and where a request cannot be honoured, the reason is recorded in `stokes.pc_fallbacks` instead of the solve quietly running on something else. `stokes.preconditioner_settings` reports what was actually applied.

4. CURVED BOUNDARIES HAVE TO BE TOLD THEY ARE CURVED

Refining a mesh puts new nodes at the midpoints of existing edges. On a straight boundary that is fine. On a curved one it is not: the coarse mesh approximates a circle by a polygon, and the midpoint of a chord lies inside the circle, not on it. Refine naively and every level keeps the coarsest level's polygon, so the geometry never improves no matter how fine the mesh becomes.

Underworld's curved meshes carry a refinement callback that snaps boundary-labelled nodes back onto the true surface after each refinement. For the annulus it is a few lines — take the nodes labelled `Upper` and `Lower` and rescale each to the correct radius:

```
coords[upper] *= radiusOuter / R[upper]
coords[lower] *= radiusInner / R[lower]
```

This happens automatically for the built-in meshes, so in normal use there is nothing to do. It matters if you build a mesh of your own with curved boundaries: without a callback of this kind the hierarchy is geometrically wrong, and the error does not go away with refinement.

5. WHAT THE COARSE MESH LEAVES BEHIND

Because the fine mesh is made by subdividing the coarse one, the coarse mesh's own triangulation is still visible in it. Its edges survive as continuous lines across the fine mesh, and its vertices remain places where an unusual number of elements meet. The pattern is a record of how the mesh was made rather than of the problem being solved.

`mesh.relax()` moves the nodes to improve element shapes while keeping the resolution and the topology, which loosens that imprint. On a twice-refined box it lifts median element quality from 0.976 to 0.994. The gain is small there because a gmsh base mesh is already good; it is larger the worse the base is. Relaxation moves coordinates only, so the hierarchy survives it — that is the subject of a [companion note](#).

6. WHEN THE CHOICE MATTERS

The same box, the same discretisation, twice-refined so a hierarchy exists, and only two things varied: whether there is a viscosity contrast, and which preconditioner the velocity block uses.

viscosity contrast	preconditioner	velocity iterations	outcome	wall clock
1	GAMG	391	converged	3.3 s
1	FMG	2	converged	2.0 s
10^4	GAMG	2000	DIVERGED_ITS	78 s
10^4	FMG	3	converged	3.0 s

On the constant-viscosity problem both work. GAMG takes a few hundred iterations of the velocity block against two, but the solve is a couple of seconds either way, and on a model that size the choice does not change the day.

With a factor of 10^4 across a viscous layer, GAMG does not converge the velocity block at all: it runs into the iteration cap, at 200 and again at 2000, and takes 78 seconds to fail. FMG takes three iterations and three seconds. The gap is not a speed-up, it is the difference between a model that runs and one that does not.

The GAMG here is the default configuration

These runs use the GAMG bundle Underworld applies without tuning. Algebraic multigrid on an elasticity-like operator usually wants the near-null-space — the rigid body modes — supplied to it, and it can be made considerably better than this. The comparison shows what the two choices give you out of the box, which is the choice most people are actually making.

7. USING IT

```
# Build the hierarchy at mesh construction: base + two refinements.
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)

stokes = uw.systems.Stokes(mesh)
stokes.preconditioner = "auto"          # FMG when a hierarchy exists
stokes.solve()

print(len(mesh.dm_hierarchy))          # how many levels there are
```

```
print(stokes.preconditioner_settings) # what was applied  
print(stokes.pc_fallbacks)           # anything declined, and why
```

If a solve is slow and the mesh has no hierarchy, that is the first thing to change: rebuild the mesh with `refinement` rather than at a fine `cellSize`, and the same resolution arrives with the coarse levels attached.